

SUMMARY

보안 로그 분석 도메인에서 백엔드, 데이터 파이프라인, Kubernetes 운영 인프라를 함께 다뤄 온 백엔드 엔지니어입니다. Go 기반 멀티테넌트 수집 플랫폼을 설계·구축하고, Kafka/Redis/PostgreSQL/OpenSearch를 이용해 데이터 수집, 정규화 흐름을 운영했습니다. 다양한 기능 구현과 함께 CI/CD를 통한 개발 속도 개선, 운영 문서화까지 제품 운영에 필요한 기반을 함께 개선해 왔습니다.

1. 보안 로그 수집·분석 플랫폼

(주)인브릿지

2025.06 - 현재

프로젝트 개요: 보안 이벤트 로그를 수집·정규화하고, 검색 엔진 기반 탐지·분석 기능을 제공하는 플랫폼.

역할: 백엔드 및 데이터 파이프라인 설계와 구축, Kubernetes 운영 인프라 구축 및 운영, CI/CD 및 운영 안정화 주도.

기술 스택: Next.js, NestJS, Go, PostgreSQL, Redis, Kafka, OpenSearch, AWS, Kubernetes

대표 성과:

- 인증·테넌트 등 전 기능이 공유하는 기반을 중심으로 도메인 기능을 확장할 수 있는 백엔드 Hub & Spoke 아키텍처 설계
- 마이크로서비스 기반 수집 플랫폼 구축하고, Adapter 패턴을 기반으로 신규 연동 대상 추가 비용을 축소
- 다양한 외부 데이터 소스를 연동하고, 워커 상태 감시와 자동 복구로 수집 연속성 확보
- Kubernetes 기반 운영 인프라 구축
- 여러 서비스가 공유하는 공용 CI 파이프라인을 설계해 통합 배포 과정을 표준화

1.1 백엔드 Hub & Spoke 아키텍처 설계

문제: 멀티테넌트 서비스가 공통으로 요구하는 기반 기능과, 요구사항에 따라 자주 변하는 도메인 기능이 한 백엔드에 함께 쌓이고 있었습니다. 이를 계속 누적하면 공통 정책과 도메인 로직이 기능별 구현에 흩어지고, 반대로 서비스를 너무 잘게 나누면 개발 속도가 느려질 위험이 있었습니다.

접근:

- 인증, 테넌트 컨텍스트, 권한, 공통 도메인 계약처럼 전 기능이 공유하는 기반을 Hub로 묶고, 변경이 잦은 도메인 기능은 Spoke로 분리
- Hub는 API 계층과 공통 도메인 모델을 담당하고, Spoke는 독립적으로 배포·확장할 수 있는 실행 단위로 설계
- 공통 인증, 테넌트, 운영 정책은 Hub에서 일관되게 적용되도록 설계

결과:

- Hub의 API 문서를 통해 프론트엔드의 연동 지점을 통합하고, 백엔드 구현 변경이 프론트엔드에 미치는 영향을 줄임
- 신규 도메인 기능을 기존 API/운영 정책을 흔들지 않고 Spoke 단위로 추가할 수 있는 구조 확보
- 공통 기능을 Hub에 집중시켜 Spoke별 중복 구현을 줄이고, 운영 정책 적용 지점을 통합
- API 계층과 실행 계층의 경계를 분리해 기능 변경 시 영향 범위와 배포 리스크를 축소

1.2 멀티테넌트 데이터 수집 플랫폼 설계·구축

문제: 연동 대상마다 API와 수집 방식, 재시도 정책이 달라 신규 대상을 추가할 때마다 수집 로직이 중복되고 운영 기준이 흔들릴 위험이 있었습니다.

접근:

- 수집 제어 계층과 실행 계층을 분리한 마이크로서비스 구조로 설계
- 연동 대상마다 다른 수집 방식을 소수의 실행 모델로 분류하고, 공통 인터페이스 계약을 정의해 모든 구현이 이를 따르도록 통합
- 상태 관리, 실행, 이벤트 전달의 책임을 분리해 재시도와 확장 지점을 명확히 구분

결과:

- 서로 다른 수집 방식을 공통 인터페이스로 일반화해 연동 확장이 가능한 구조 확보
- 수집 도메인 모델을 일관된 형태로 정리
- 연동 대상별 API 차이를 Adapter 패턴으로 격리해 신규 추가 시 기존 로직 변경과 리스크를 축소

1.3 운영 인프라와 공용 CI/CD 파이프라인

문제: 여러 서비스에 공통 적용되는 배포 표준과 재현 가능한 파이프라인이 필요했습니다.

접근:

- Kubernetes 클러스터를 구축하고, 공통 플랫폼 계층을 표준화해 모든 서비스가 동일한 기반 위에서 동작하도록 구성
- 여러 서비스가 공유하는 공용 CI 파이프라인 설계

결과:

- 운영 환경에 가용성 확보 구성을 적용해 장애 대응 기반 마련
- 서비스와 환경 전반에 공용 CI/CD를 적용해 관리 부담을 축소
- 배포 파이프라인 운영 절차를 문서화해 전 구간에 일관 적용

2. 통화·회의 녹취 및 AI 요약 서비스

(주)인브릿지

2025.03 - 2025.05

프로젝트 개요: 통화·회의 녹취록을 자동 생성하고 AI로 요약하는 서비스.

역할: 어드민 페이지, 백엔드 API, 개발 프로세스 담당.

기술 스택: React, JavaScript, Node.js, Express, PostgreSQL, Docker, GitHub

대표 성과:

- GitHub 브랜치/PR 기반 협업 체계와 코드 리뷰 절차를 담당해 변경 이력을 추적 가능한 상태로 운영
- Docker 기반 개발 환경을 구성해 OS와 무관한 실행 환경과 로컬 디버깅 재현성 확보
- 도메인 모델 구조를 단순화하는 RFC를 작성하고, API 정렬·페이지네이션 규칙을 표준화
- React 어드민 화면과 Node.js/Express API를 개발하며 프론트엔드와 백엔드 간 데이터 계약을 정리

2.1 RFC 작성과 API/데이터 모델 정리

문제: API와 프론트엔드에서 일관된 형태로 표현할 도메인 모델 정리가 필요했고, 동시에 협업 규칙과 개발 환경 표준을 함께 정의해야 했습니다.

접근:

- 도메인 모델 단순화 방안을 RFC로 제안
- API 엔드포인트와 페이지네이션 규칙을 표준화해 프론트엔드와 백엔드 간 데이터 계약을 명확히 정리
- GitHub 브랜치/PR 기반 협업 체계와 리뷰 절차를 담당해 변경 이력을 추적 가능한 상태로 운영
- Docker 기반 개발 환경을 구성해 OS와 무관한 실행 환경과 로컬 디버깅 재현성 확보

결과:

- 도메인 모델과 API·프론트엔드 간 데이터 계약이 정리되어 개발 효율성 향상
- 변경 이력과 리뷰 기록이 남는 협업 체계로 안정적인 개발 흐름 확보
- OS와 무관한 실행 환경과 로컬 디버깅 재현성을 확보해 개발 속도 향상